

README – Unit 4: Fundamentals of ANN

Artificial Neural Networks (ANN)

This repository contains complete notes, theory, numericals, Python implementations, mini projects, viva questions, and mind maps for **Unit 4: Fundamentals of ANN**.

Topics Covered

- Understanding Neural Network
 - Basics of ANN
 - Single Perceptron
 - Perceptron Learning Rule
 - Activation Functions
 - Binary Classification using Perceptron
 - Multi Layer Perceptron (MLP)
 - RBFClassifier
 - ANN Simulations using scikit-learn
 - Student Performance Prediction
 - Music Genre Classification
 - Handwritten Digit Recognition using MLP
-

1. Understanding Neural Network

Artificial Neural Network is a Machine Learning model inspired by the human brain.

Neural Networks consist of:

- Input Layer
- Hidden Layer
- Output Layer

Each neuron:

1. Takes input
 2. Multiplies with weights
 3. Adds bias
 4. Applies activation function
 5. Produces output
-

ANN Formula

$$Y = f(\sum wx + b)$$

Where:

- x = input
- w = weight
- b = bias
- f = activation function

2. Basics of ANN

Term	Meaning
Neuron	Basic processing unit
Weight	Importance of input
Bias	Extra adjustable value
Epoch	One complete training cycle
Learning Rate	Controls learning speed

3. Single Perceptron

Perceptron is the simplest neural network model.

Used for:

- Binary Classification
- AND Gate
- OR Gate

Perceptron Formula

$$z = w_1x_1 + w_2x_2 + b$$

Output:

$$Y = f(z)$$

4. Perceptron Learning Rule

Weights update formula:

$$w(\text{new}) = w(\text{old}) + \eta(t - o)x$$

Where:

- η = learning rate
- t = target output
- o = predicted output

5. Activation Functions

Step Function

$$\begin{aligned} f(x) &= 1, \quad x \geq 0 \\ f(x) &= 0, \quad x < 0 \end{aligned}$$

Use

Binary classification.

Sigmoid Function

$$\sigma(x) = 1 / (1 + e^{-x})$$

Output Range

0 to 1

Use

Probability prediction.

ReLU Function

$$f(x) = \max(0, x)$$

Advantage

Fast training.

Tanh Function

Output range:

-1 to 1

6. Binary Classifier using Perceptron

AND Gate Example

x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Python Code

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,0,0,1])

model = Perceptron()
model.fit(X,y)

print(model.predict([[1,1]]))
```

7. Multi Layer Perceptron (MLP)

MLP stands for Multi Layer Perceptron.

It contains:

- Input Layer
- Hidden Layers
- Output Layer

Used for:

- Deep Learning
- Pattern Recognition
- Complex Predictions

Features of MLP

- Learns non-linear patterns
- Uses backpropagation
- High accuracy

MLP Implementation

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.2
)

model = MLPClassifier(
    hidden_layer_sizes=(10,10),
    max_iter=1000
)

model.fit(X_train,y_train)

print(model.score(X_test,y_test))
```

8. RBFClassifier

RBF = Radial Basis Function.

Used in:

- Pattern Recognition
- Classification
- Function Approximation

RBF Formula

$$\phi(x) = e^{-\beta ||x - c||^2}$$

Where:

- c = center
- β = spread parameter

RBF Implementation

```
from sklearn import svm
from sklearn.datasets import load_iris

iris = load_iris()

X = iris.data
y = iris.target

model = svm.SVC(kernel='rbf')
model.fit(X,y)

print(model.predict([X[0]]))
```

9. Simulations using scikit-learn

MLP Simulation

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
```

```
        hidden_layer_sizes=(50,),  
        max_iter=500  
    )
```

RBF Simulation

```
from sklearn.svm import SVC  
  
model = SVC(kernel='rbf')
```

10. Student Performance Prediction using MLP

Features Used

- Study Hours
- Attendance
- Assignments

Python Example

```
from sklearn.neural_network import MLPClassifier  
  
X = [  
    [2,60],  
    [5,90],  
    [1,40],  
    [6,95]  
]  
  
y = [0,1,0,1]  
  
model = MLPClassifier(max_iter=1000)  
  
model.fit(X,y)  
  
print(model.predict([[4,80]]))
```

11. Music Genre Classification using RBFClassifier

Genres

- Rock
 - Pop
 - Jazz
 - Classical
-

Features Used

- Tempo
 - Pitch
 - Frequency
 - Beat
-

Workflow

Audio File → Feature Extraction → RBFClassifier → Genre Prediction

12. Mini Project – Handwritten Digit Recognition using MLP

Goal

Recognize handwritten digits from 0–9.

Dataset Used:

- MNIST
 - Digits Dataset
-

Python Code

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

digits = load_digits()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    digits.data,
    digits.target,
    test_size=0.2
)

model = MLPClassifier(
    hidden_layer_sizes=(64,64),
    max_iter=1000
)

model.fit(X_train,y_train)

print(model.score(X_test,y_test))
```

Difference Between Perceptron, MLP and RBF

Feature	Perceptron	MLP	RBF
Layers	Single	Multiple	Multiple
Complex Learning	No	Yes	Yes
Activation	Step	ReLU/Sigmoid	Radial Basis
Use Case	Simple Classification	Deep Learning	Pattern Recognition

Important Viva Questions

Q1. What is ANN?

ANN is a brain-inspired ML model made of interconnected neurons.

Q2. What is perceptron?

A simple neural network used for binary classification.

Q3. What is activation function?

A function deciding whether neuron activates or not.

Q4. What is MLP?

A multi-layer neural network using hidden layers.

Q5. What is backpropagation?

Algorithm used to update weights during training.

Important Numerical

Numerical 1

Given:

- $x_1 = 1$
- $x_2 = 0$
- $w_1 = 0.5$
- $w_2 = 0.2$
- $b = 0.1$

Find output.

Solution

$$z = (0.5)(1) + (0.2)(0) + 0.1$$

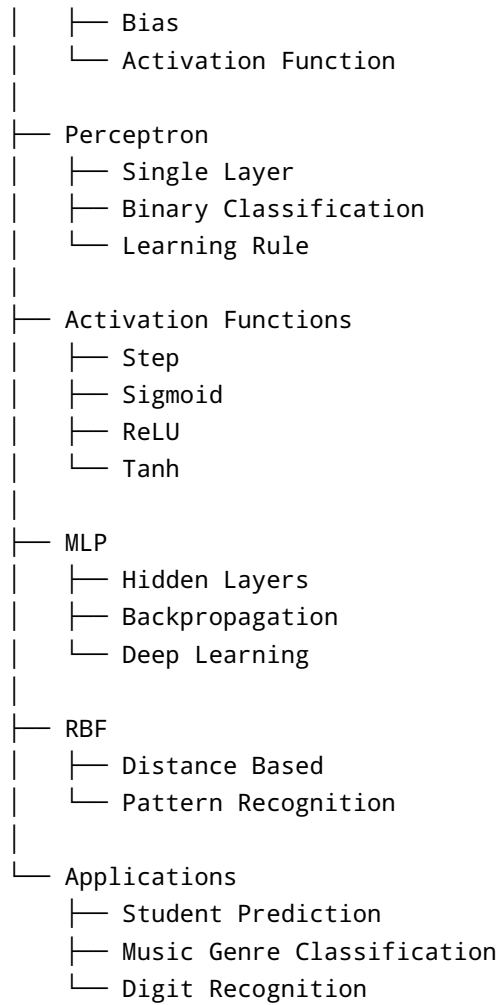
$$z = 0.6$$

Using step activation:

$$\text{Output} = 1$$

Mind Map

```
ANN
|
├─ Basics
|   ├── Neuron
|   └── Weight
```



End of Unit 4 – Fundamentals of ANN